

Exercise 4: The Data Layer: SQL Schema and ORM Models

DEV CYCLE	Develop (data)
CC FEATURES	The build loop, @file references, running code to validate, targeted single-file edits
SPEC TRACES	SPEC §5.1 tables, §5.2 invariants, FR-02 (integer doses), NFR-02 (UTC)
TIME	35 min
YOU PRODUCE	Working app/db.py and app/models.py that create the DischargeFlow schema, with database-level guards

Objective

Build the foundation: the database. You will run the Claude Code build loop for the first time (prompt, Claude writes, you run, you correct) and learn to validate every step by executing the code rather than trusting the summary. In a clinical system the schema is also your first safety layer: a dose that cannot be stored as a float cannot be rounded wrong later.

Walk-through

1 Generate the database wiring

Ask for the SQLAlchemy engine, session and base, plus a function that creates the tables. Reference the architecture so Claude builds to the agreed plan, not to its own taste.

TYPE THIS PROMPT INTO CLAUDE CODE

Implement app/db.py per @docs/ARCHITECTURE.md and @CLAUDE.md: a SQLAlchemy 2.x engine on SQLite reading DATABASE_URL from the environment (default sqlite:///dischargeflow.db), a SessionLocal factory, a DeclarativeBase, a get_db dependency for FastAPI, and init_db() that creates all tables. SQLite ignores foreign keys unless told otherwise, so enable PRAGMA foreign_keys=ON on every connection. Type hints throughout. Add an empty app/__init__.py.

VALIDATE Read the diff before approving: the engine URL comes from the environment with the right default; get_db yields and closes a session in try/finally; there is an event listener that runs PRAGMA foreign_keys=ON. That last line is the one most AI drafts forget.

SCREENSHOT SLOT *Diff of app/db.py showing the foreign_keys PRAGMA listener*

2 Generate the models from the spec

Now the tables, straight from SPEC §5.1. You are asking for seven models in one go, so be explicit about the constraints that carry safety meaning.

TYPE THIS PROMPT INTO CLAUDE CODE

Create app/models.py with SQLAlchemy 2.x typed models for every table in SPEC §5.1 except agent_memory (that comes in Exercise 10): DischargeRecord, Medication, SummaryDraft, Review, ReferenceDocument, ReferenceChunk, AuditEvent. Rules: dose_x1000 is an INTEGER with a CHECK that it is > 0 AND typeof(dose_x1000) = 'integer', because SQLite will otherwise happily store 2.5 in an INTEGER column. Status, dose_unit, route and change_type get CHECK constraints listing the allowed values from the spec. patient_ref is unique. reviews.draft_id is unique (one decision per draft, FR-11). (record_id, version) on summary_drafts is unique. Timestamps default to UTC now. JSON columns are TEXT. Cite the SPEC section in the module docstring.

VALIDATE Scan the diff for each rule: the typeof CHECK on dose_x1000, the enum CHECKs, the three unique constraints, UTC defaults (datetime.now(timezone.utc), not the deprecated utcnow()),

and relationships with cascade delete from record to medications. If a constraint is missing, say which one; do not accept "it is validated in pydantic" for this layer.

3 Create the tables and prove it

Code that has not run is a guess. Initialise the database yourself.

RUN IN TERMINAL

```
PS> python -c "from app.db import init_db; init_db(); print('tables created')"
```

VALIDATE You see `tables created` and a `dischargeflow.db` file appears in the repo root. A `ModuleNotFoundError` for `app` usually means a missing `app/__init__.py` or running from the wrong directory.

4 Inspect the real schema

Trust, then verify. Ask the database itself what it created, not Claude. Python 3.12 ships a small `sqlite3` command line, so this works the same on every OS. The quoting rule for today: outer double quotes, inner single quotes.

RUN IN TERMINAL

```
PS> python -m sqlite3 dischargeflow.db "SELECT name FROM sqlite_master WHERE type='table'"
PS> python -m sqlite3 dischargeflow.db "SELECT sql FROM sqlite_master WHERE
name='medications'"
```

VALIDATE The first command lists all seven tables. The second prints a `CREATE TABLE` for `medications` containing `dose_x1000 INTEGER NOT NULL`, the `typeof CHECK`, and the enum `CHECKs`. This is ground truth from the database. On Python 3.10/3.11 ask Claude for an equivalent one-off script.

SCREENSHOT SLOT *CREATE TABLE medications output showing the CHECK constraints*

5 Prove the dose guard

A constraint you have not seen fail is a constraint you are only hoping works. Ask Claude to demonstrate the failure by running it.

TYPE THIS PROMPT INTO CLAUDE CODE

Without changing any files, write and run a one-off Python snippet that creates a `DischargeRecord` and tries to add a `Medication` with `dose_x1000=2.5`. Then try `dose_x1000=0`. Show me the exact exception for each, then roll back.

VALIDATE Both inserts fail with `IntegrityError: CHECK constraint failed`. Claude asked permission before running the snippet and did not edit any file. You have now proven, not assumed, FR-02 at the storage layer.

What success looks like

- `dischargeflow.db` contains the seven tables from SPEC §5.1, and their constraints match the spec.
- You validated the schema by querying the database directly, not by reading Claude's summary.
- You watched a float dose and a zero dose get rejected by the database itself.

Common pitfalls (Windows and macOS)

- SQLite type affinity: an `INTEGER` column accepts 2.5 unless you add the `typeof CHECK`. This is the kind of "confident wrongness" you will meet again in Exercise 9.

- Foreign keys are off by default in SQLite. Without the PRAGMA, deleting a record leaves orphan medications and nobody notices.
- If you change a model after the first `init_db()`, `create_all` will not alter an existing table. For the PoC, delete `dischargeflow.db` and re-run; production needs migrations (Exercise 11 gap audit).

Go deeper • AI developer track

Optional, for faster finishers. Each challenge extends the lab with a production-grade technique.

- Enable PRAGMA `journal_mode=WAL` alongside the foreign-key PRAGMA and explain, with the concurrent-append scenario from Ex 3, what WAL changes and what it does not guarantee.
- Write `scripts/check_schema.py` that compares the ORM metadata with `sqlite_master` and exits non-zero on drift. This is the seed of the migrations story in the gap audit.

Stretch goal

Ask Claude to add `scripts/seed.py` that loads every JSON file in `samples/records/` into the database through the ORM, then run it and count rows in `discharge_records` and `medications` with `python -m sqlite3`. You now have realistic data for the next exercise.